

PROJETO 1: FAMILIARIZAÇÃO COM MÁQUINA VIRTUAL E SINCRONIZAÇÃO DE PROCESSOS

Izaque Vieira de Souza¹, Olávio Gonçalves de Almeida¹, Wisley Kilder da Silva Nogueira¹

¹Departamento de Ciência da Computação – Universidade Federal de Goiás (UFG)
izaquevieira@outlook.com.br, olaviog.almeida@gmail.com,
w.kilder@hotmail.com

1. Tarefa 1

1.1. Descrição

Devido a questões de mobilidade e praticidade, a equipe de trabalho optou por utilizar um computador próprio para realizar as atividades propostas na disciplina. Na máquina utilizada para os trabalhos, o SO Ubuntu 16.04 LTS foi instalado de forma nativa, com o tamanho de disco destinado ao SO de 109 GB, e uma memória de 7,7 GB.

1.2. Instalação de Softwares

Os softwares para essa distribuição Linux podem ser instalados através da loja Ubuntu Software, ou através da interface de linha de comando, utilizando comandos no terminal.

Na máquina utilizada, a maioria dos softwares instalados foram através da interface de linha de comando. Em muitos dos casos foi necessário incluir o repositório do software para que a instalação fosse possível.

2. Tarefa 2

2.1. Sincronização de processos

Essa tarefa tem como o objetivo recapitular conceitos fundamentais de sistemas operacionais, vistos em SO1. Nas seções a seguir serão apresentados os códigos desenvolvidos pelo grupo.

2.1.1. Solução de Peterson

Para a resolução dessa atividade foram usadas como base os códigos apresentados pelo professor nos slides.

1) A seção crítica foi protegida de acordo com o que foi pedido no enunciado, utilizando a proposta apresentada.

2) A solução de Peterson protege a seção crítica da seguinte forma: o código fica preso em um laço de repetição enquanto um processo está na sua seção crítica (SC), e

quando ele libera a SC o laço de repetição é quebrado, liberando acesso à SC para outros processos que tenham interesse.

3) O tratamento de erros pode ser feito com um condicional *if*. Se a função utilizada para criar *threads* retornar 0, a *thread* foi criada corretamente. Caso contrário, a *thread* não foi criada corretamente, e o programa encerra.

4) O código apresentado a seguir foi adaptado para criar duas *threads* p_i e p_j utilizando apenas a seção de código da função *void * cria(void *arg)*. Para realizar essa adaptação foi utilizada a documentação da função *pthread_create()*, apresentada por Kerrisk (2018).

Código:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define true 1
#define false 0

pthread_t flag[2];
int turn, counter = 0;

void enterSC (int i){
    int j = 1 - i; //J sempre vai ser oposto de i: caso i == 0, j == 1 e vice-versa
    flag[i] = true; //Thread que solicita entrada na seção crítica
    turn = j;
    while(flag[j] && turn == j); //Laço para segurar a seção crítica
    return;
}

void leaveSC (int i){
    flag[i] = false; //Thread que solicita sair da seção crítica
    return;
}

void * cria (void *arg){
```

```

int aux = (int *)arg, i = 0; //Pegar o índice da thread
enterSC(aux); //Chamada da função para entrar na seção crítica
printf("\nP%d: A região crítica é minha!!!\n", aux);
counter++;
printf("\nP%d: counter = %d\n\n", aux, counter);
leaveSC(aux); //Função para sair da seção crítica
return NULL;
}

```

```

int main (void){
    pthread_t thi, thj; //Duas variáveis para a criação das threads

    //Verifica se as threads foram criadas corretamente
    if(pthread_create (&thi, NULL, cria, (int *)0) != 0){
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }
    if(pthread_create (&thj, NULL, cria, (int*)1) != 0){
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }

    //Inicia as threads
    pthread_join (thi, NULL);
    pthread_join (thj, NULL);

    return 0;
}

```

2.1.2. Solução de hardware usando locks do tipo test and set

Para realizar esta atividade foram usados como base o material disponibilizado pelo professor e também uma breve descrição das funções *atomic_flag_test_and_set* e *atomic_flag_clear*, apresentadas em uma *homepage* (*atomic_flag_test_and_set*, *atomic_flag_clear* – cppreference.com).

Código:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdatomic.h> //Biblioteca atomic

int counter;
atomic_flag lock = ATOMIC_FLAG_INIT; //Variável atômica

void enterSC (int i){
while(atomic_flag_test_and_set(&lock)); //Laço para travar a SC
return;
}

void leaveSC (int i){
atomic_flag_clear(&lock); //Libera a seção crítica
return;
}

void * cria (void *arg){
int aux = (int *)arg, i = 0; //Pegar o índice da thread
enterSC(aux); //Chamada da função para entrar na seção crítica
printf("\nP%d: A região crítica é minha!!!\n", aux);
counter++;
printf("\nP%d: counter = %d\n\n", aux, counter);
leaveSC(aux); //Função para sair da seção crítica
return NULL;
}

int main (void){
pthread_t thi, thj; //Duas variáveis para a criação das threads

//Verifica se a thread foi criada corretamente
```

```

if(pthread_create (&thi, NULL, cria, (int *)0) != 0){
    printf("Algum erro ocorreu na criação das threads!\n");
    exit(1);
}

//Verifica se a thread foi criada corretamente
if(pthread_create (&thj, NULL, cria, (int*) 1) != 0){
    printf("Algum erro ocorreu na criação das threads!\n");
    exit(1);
}

//Inicia as threads
pthread_join (thi, NULL);
pthread_join (thj, NULL);

return 0;
}

```

2.1.3. Solução de hardware usando locks do tipo mutex

Para solucionar o problema proposto foi usado como base o material disponibilizado pelo professor (slides).

Código:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t lock; //Variável para travamento da SC
int counter = 0; //Contador

void enterSC (){
    pthread_mutex_lock(&lock); //Trava a seção crítica pela variável lock
    return;
}

```

```
void leaveSC (){
    pthread_mutex_unlock(&lock); //Libera a SC pela variável lock
    return;
}
```

```
void * cria (void *arg){
    int aux = (int *)arg, i = 0; //Pegar o índice da thread
    enterSC(); //Chamada da função para entrar na seção crítica
    printf("\nP%d: A região crítica é minha!!!\n", aux);
    counter++;
    printf("\nP%d: counter = %d\n\n", aux, counter);
    leaveSC(); //Função para sair da seção crítica
    return NULL;
}
```

```
int main (void){
    pthread_t thi, thj; //Duas variáveis para a criação das threads

    //Verifica se a thread foi criada corretamente
    if(pthread_create (&thi, NULL, cria, (int *)0) != 0){
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }

    if(pthread_create (&thj, NULL, cria, (int*) 1) != 0){
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }

    //Inicia as threads
    pthread_join (thi, NULL);
    pthread_join (thj, NULL);
}
```

```
    return 0;
}
```

2.1.4. Solução de hardware usando semáforo

Para a resolução dessa atividade foram utilizados como referência o material disponibilizado pelo professor, com os slides da disciplina de SO1. Foi usada também uma descrição das funções de semáforo disponíveis em uma *homepage* (sem_wait, sem_post, sem_init, sem_destroy - man7.org).

Código:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h> //Biblioteca para o uso de semáforos

sem_t semaphore; //Variável de semáforo
int counter = 0;

void enterSC () { //Função para controlar entrada na seção crítica
    sem_wait(&semaphore); //Para fazer o lock do semáforo
    return;
}

void leaveSC () { //Função para sair da seção crítica
    sem_post(&semaphore); //Função que libera o semáforo
    return;
}

void * cria (void *arg) { //Função para a thread
    int aux = (int *)arg, i = 0; //Pegar o índice da thread
    enterSC(); //Chamada da função para entrar na seção crítica
    printf("\nP%d: A região crítica é minha!!!\n", aux);
    counter++;
    printf("\nP%d: counter = %d\n\n", aux, counter);
}
```

```

leaveSC(); //Função para sair da seção crítica
return NULL;
}

int main (void){
    pthread_t thi, thj; //Duas variáveis para a criação das threads
    sem_init(&semaphore, 0, 1); //Função para inicializar a variável de semáforo

    //Verifica se a thread foi criada corretamente
    if(pthread_create (&thi, NULL, cria, (int *)0) != 0){
        printf("Alguns erros ocorreram na criação das threads!\n");
        exit(1);
    }

    if(pthread_create (&thj, NULL, cria, (int*) 1) != 0){
        printf("Alguns erros ocorreram na criação das threads!\n");
        exit(1);
    }

    //Inicia as threads
    pthread_join (thi, NULL);
    pthread_join (thj, NULL);

    //Destroi a variável do semáforo ao fim do programa
    sem_destroy(&semaphore);
    return 0;
}

```

3. Tarefa 3

Para a resolução desta atividade foi usado o código de semáforos apresentado na seção 2.1.4. Na prática, é feita uma simulação onde dois processos são executados simultaneamente. Um processo p_i solicita um recurso Q , e ao mesmo tempo um outro processo p_j solicita um recurso S . Após isto, ambos os processos dormem por 1 segundo e posteriormente p_i solicita o recurso S e p_j solicita o recurso Q . Como ambos os recursos já estavam alocados, o programa entra em um deadlock. O código feito é apresentado em seguida.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>
```

```
sem_t semaphore, semaphore2; //Variáveis do tipo semáforo
int counter = 0; //Variável contadora
```

```
void enterSCQ(int i){ //Função para pegar o recurso Q
    sem_wait(&semaphore);
    printf("O processo %d pegou o recurso Q e aguarda pelo recurso S!!\n", i);
    return;
}
```

```
void leaveSCQ (){
    sem_post(&semaphore);
    return;
}
```

```
void enterSCS(int i){ //Função para pegar o recurso S
    sem_wait(&semaphore2);
    printf("O processo %d pegou o recurso S e aguarda pelo recurso Q!!\n", i);
    return;
}
```

```
void leaveSCS (){
    sem_post(&semaphore2);
    return;
}
```

```
void * pi (void *arg){
    int aux = (int *)arg; //Índice
```

```

enterSCS(aux); //Tenta alocar o recurso S
sleep(1);
enterSCQ(aux); //Tenta alocar o recurso Q
printf("\nPi: A região crítica é minha!!!\n");
counter++;
printf("\n\nPi: counter = %d\n\n", counter);
leaveSCS();
leaveSCQ();
return NULL;
}

```

```

void * pj (void *arg){
    int aux = (int *)arg; //Índice
    enterSCQ(aux); //Tenta alocar o recurso Q
    sleep(1);
    enterSCS(aux); //Tenta alocar o recurso S
    printf("\nPj: A região crítica é minha!!!\n");
    counter++;
    printf("\n\nPj: counter = %d\n\n", counter);
    leaveSCQ();
    leaveSCS();
    return NULL;
}

```

```

int main (void){
    pthread_t thi, thj; //Variáveis para a criação de threads

    //Iniciação de semáforos
    sem_init(&semaphore, 0, 1);
    sem_init(&semaphore2, 0, 1);

    //Verificação se as threads foram criadas corretamente
    if(pthread_create (&thi, NULL, pi, (int *)0) != 0){
        printf("Algum erro ocorreu na criação das threads!\n");
    }
}

```

```

        exit(1);
    }
    if(pthread_create (&thj, NULL, pj, (int*)1) != 0){
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }

    //Iniciando as threads
    pthread_join (thi, NULL);
    pthread_join (thj, NULL);

    //Destruindo os semáforos
    sem_destroy(&semaphore);
    sem_destroy(&semaphore2);

    return 0;
}

```

4. Tarefa 4

A tarefa proposta é apresentar uma implementação do algoritmo do jantar dos filósofos, utilizando *threads* e semáforos. A implementação feita utiliza um semáforo para cada filósofo, e também um vetor indicando o status de cada filósofo (Pensando, Faminto, Comendo). Sempre que um filósofo está no estado “Faminto” ele verifica se o garfo à sua esquerda e direita estão disponíveis. Caso sim, ele tem condições de comer, e caso contrário ele aguarda até que os garfos estejam disponíveis. O código será apresentado em seguida, e quando executado apresenta detalhadamente a situação de cada filósofo e de cada garfo.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

#define N 5    //Número de filósofos
#define LEFT (i+N-1)%N    //Filósofo à esquerda de i

```

```

#define RIGHT (i+1)%N    //Filosofo à direita de i
#define THINKING 0
#define HUNGRY 1
#define EATING 2

pthread_t tid[N];    //Uma thread para cada filósofo
int state[N];    //Estado de cada filósofo
sem_t mutex;
sem_t s[N];    //Um semáforo para cada filósofo

void test(int i){
    if(state[LEFT] == 0)
        printf("O filosofo a esquerda de %d está pensando!!\n", i+1);
    else if (state[LEFT] == 1)
        printf("O filosofo a esquerda de %d está com fome!!\n", i+1);
    else if (state[LEFT] == 2)
        printf("O filosofo a esquerda de %d está comendo!!\n", i+1);

    if(state[RIGHT] == 0)
        printf("O filosofo a direita de %d está pensando!!\n", i+1);
    else if (state[RIGHT] == 1)
        printf("O filosofo a direita de %d está com fome!!\n", i+1);
    else if (state[RIGHT] == 2)
        printf("O filosofo a direita de %d está comendo!!\n", i+1);

    //Testa se o filósofo pode pegar os garfos
    if(state[i]==HUNGRY && state[LEFT]!=EATING && state[RIGHT]!=EATING){
        state[i] = EATING;
        printf("O filosofo %d pegou os garfos %d e %d\n", i+1, LEFT+1, i+1);
        sem_post(&s[i]);
    }
}

void take_forks(int i){    //Função para pegar os garfos

```

```

sem_wait(&mutex);      //Trava o semáforo mutex
state[i] = HUNGRY;     //Muda o estado do filósofo para "Faminto"
printf("O filosofo %d está com fome!!\n", i+1);
test(i);              //Chama a função para testar a situação dos garfos à direita e esquerda
sem_post(&mutex);      //Libera o semáforo mutex
sem_wait(&s[i]);       //Trava o semáforo do filósofo i
}

void put_forks(int i){ //Função para devolver os garfos
    sem_wait(&mutex); //Trava o semáforo mutex
    state[i] = THINKING; //Altera o estado do filósofo i para "Pensando"
    printf("O filosofo %d devolveu os garfos %d e %d!!\n", i+1, LEFT+1, i+1);
    printf("O filósofo %d está pensando!!\n", i+1);
    test(LEFT); //Verifica a situação do filósofo à esquerda
    test(RIGHT); //Verifica a situação do filósofo à direita
    sem_post(&mutex); //Libera o semáforo mutex
}

void *philosopher(void *arg){ //Função do filósofo
    int i = (int *) arg; //Índice do filósofo
    while(1){
        sleep(2);
        take_forks(i); //O filósofo tenta pegar os garfos
        printf("O filosofo %d comeu.\n", i+1); //O filósofo conseguiu os garfos para comer
        put_forks(i); //O filósofo i devolve os garfos
    }
}

int main (void){
    int i;
    sem_init(&mutex, 0, 1); //Iniciando a variável de semáforo mutex

    //Iniciando as variáveis de semáforo para cada filósofo
    for(i = 0; i < N; i++)

```

```

sem_init(&s[i], 0, 0);

//Criando uma thread para cada filósofo
for(i=0;i<N;i++)
    pthread_create (&(tid[i]), NULL, philosopher, (int *)i);

//Iniciando cada thread criada
for(i=0;i<N;i++)
    pthread_join (tid[i], NULL);

//Ao final, destruindo os semáforos criados anteriormente
for(i=0;i<N;i++)
    sem_destroy(&s[i]);
    sem_destroy(&mutex);

return 0;
}

```

Referências

- ATOMIC_FLAG_CLEAR, atomic_flag_clear_explicit. Disponível em:
 <https://en.cppreference.com/w/c/atomic/atomic_flag_clear>. Acesso em: 28 ago. 2018.
- ATOMIC_FLAG_TEST_AND_SET, atomic_flag_test_and_set_explicit. Disponível em:
 <https://en.cppreference.com/w/c/atomic/atomic_flag_test_and_set>. Acesso em: 28 ago. 2018.
- KERRISK, Michael. PTHREAD_CREATE. Disponível em:
 <http://man7.org/linux/man-pages/man3/pthread_create.3.html>. Acesso em: 28 ago. 2018.
- SEM_DESTROY. Disponível em:
 <http://man7.org/linux/man-pages/man3/sem_destroy.3.html>. Acesso em: 28 ago. 2018.
- SEM_INIT. Disponível em: <http://man7.org/linux/man-pages/man3/sem_init.3.html>. Acesso em: 28 ago. 2018.
- SEM_POST. Disponível em:
 <http://man7.org/linux/man-pages/man3/sem_post.3.html>. Acesso em: 28 ago.

2018.

SEM_WAIT. Disponível em:

<http://man7.org/linux/man-pages/man3/sem_wait.3.html>. Acesso em: 28 ago. 2018.

TAVARES, Dalton Matsuo. **Sistemas Operacionais 1**: Catalão: 2018. 79 slides, color.